

AI Engineering Mindset for Forward Deployed Engineers

From Apps to Agents to Intelligence

Practical Study Guide

IIT Roorkee | Futureense.uni

1. Introduction

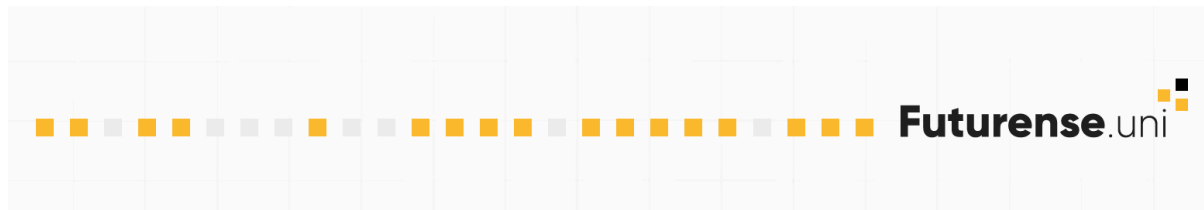
After reading this guide, you can explain what makes AI systems intelligent, choose the right model for a use case, and describe the role of a Forward Deployed Engineer with confidence.

Before AI, engineers built features. A button triggered a function, a function updated a database. That world was deterministic. AI changes the equation. Models generate probabilistic outputs, and the difference between a useful system and a failing one lies not in the model itself, but in what you build around it.

This guide covers the mental shift every AI engineer needs to make. It builds from first principles to practical frameworks.

Topics this guide covers:

- Speed vs. velocity: why direction matters as much as pace in the AI era
- The App to Agent to Intelligence evolution and what it means for your career
- Three tenets of AI intelligence: Data, Context, and Reasoning
- Evals: how to select and evaluate models for specific use cases
- Unit of Intelligence: the new unit of professional output
- The FDE role: what it is, where it came from, and what it demands
- System thinking: designing for failure, not just for the happy path



Key insight: After this guide: You can define AI intelligence precisely, explain why agents are just a wrapper, and design an AI system around the 80% that matters.

2. Core Concepts

2.1 Speed vs. Velocity

Throughout history, humans solved problems by increasing speed. Faster horses, faster trains, faster processors. Speed is a scalar: it has magnitude but no direction.

Velocity is a vector. It has both magnitude and direction. The AI era demands velocity, not just speed. Building faster is useless if you are building in the wrong direction.

Speed = |rate of output| (no direction)

Velocity = rate of output + direction of impact

A software engineer who ships 10 features per sprint moves fast. An engineer who ships one feature that eliminates the need for 10 tickets moves with velocity. The second engineer creates more value, solves a deeper problem, and is harder to replace.

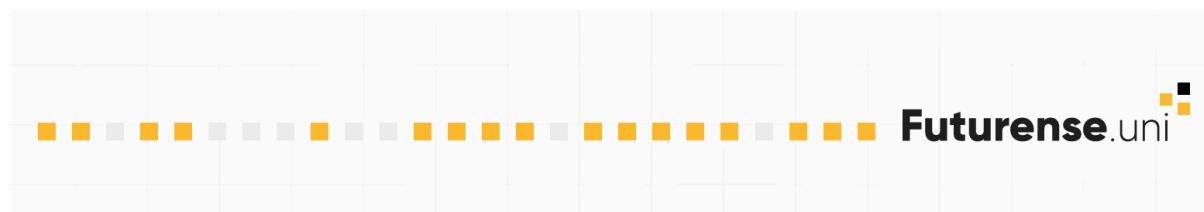
Key insight: AI handles speed. Your job is to provide direction. That is the entire shift this guide builds toward.

2.2 Deterministic vs. Probabilistic Systems

Traditional software is deterministic. If a user clicks Submit, the system writes to the database. The same input always produces the same output. Engineers could guarantee behavior with ACID properties, if-then-else logic, and unit tests.

AI systems are probabilistic. The same prompt given twice may produce two different outputs. A model generates a probability distribution over possible next tokens. The output is the highest-probability selection from that distribution. This is not a bug. It is the architecture.

Property	Deterministic System	Probabilistic System
Output for same input	Always identical	Varies by temperature and context
Failure mode	Crash, null, or exception	Hallucination, drift, or low confidence



Testing approach	Unit tests with exact assertions	Evals with statistical accuracy thresholds
Engineering guarantee	Correctness via logic	Correctness via evaluation and guardrails
What breaks systems	Bug in code	Wrong context, wrong model, missing constraints

The mental shift from deterministic to probabilistic thinking is the core challenge of AI engineering.

Watch out: Treating an LLM like a function that returns a fixed value leads to 87% of enterprise AI projects failing. The model behaves differently under different contexts, and that is expected.

3. Technical Deep Dive

3.1 The App to Agent to Intelligence Progression

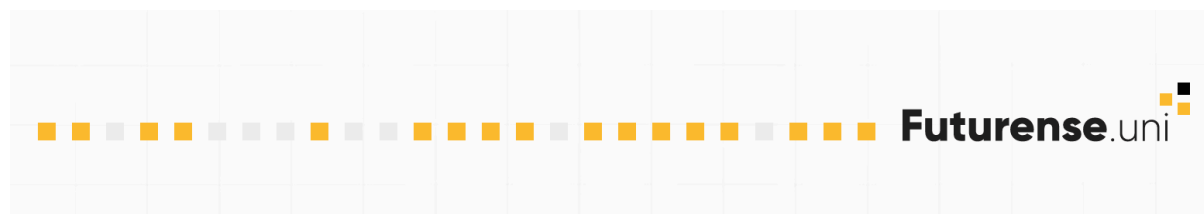
Think of this as three generations of software, each requiring a different mindset from the engineer who builds it.

The first generation was the App. A user clicked a button, the system executed a fixed function, and a result appeared on screen. The engineer's job was to write clean, bug-free, testable code. Logic lived inside if-then-else blocks and microservices. This era rewarded depth in one language or framework.

The second generation is the Agent. An agent takes an action without being explicitly asked to do so for each step. Ask it to plan a trip, and it figures out your preferences, checks availability, compares options, and delivers a result without a button click for every sub-task. It calls tools. It orchestrates. It reasons about intermediate steps.

The third generation is Intelligence. This is what customers actually pay for. Not the agent wrapper, not the code that calls the model, but the behavior the system produces under real-world conditions. Intelligence is measured by accuracy, consistency, robustness under constraint, and recovery from failure.

Generation	Unit of Work	Engineer's Focus	What Customer Pays For
App	Feature / Endpoint	Clean code, bug-free logic	Functionality delivered



Agent	Autonomous task	Orchestration, tool use, prompt design	Tasks completed without friction
Intelligence	Reliable behavior	Context, evals, failure modes, recovery	Outcomes that hold under pressure

Moving from App to Agent is a tooling upgrade. Moving from Agent to Intelligence is a mindset upgrade.

Key insight: Agent is a wrapper. Intelligence is the product. Every customer will eventually be able to buy the wrapper. None of them can buy the intelligence without the engineer who builds it.

3.2 The Three Tenets of AI Intelligence

Intelligence in an AI system comes from exactly three places. Understand these three, and you understand 80% of your job as an FDE.

Tenet 1: Data

The model has no knowledge of your specific customer, domain, or use case. All pre-training knowledge is general. Domain-specific behavior requires domain-specific data. The quality, freshness, and relevance of the data fed into or retrieved for the model determines the ceiling of intelligence possible.

Data engineering is not a back-end concern separate from AI. It is the foundation of AI engineering. Garbage data produces garbage output, regardless of model size.

Tenet 2: Context

Context is the data you give the model at inference time. It is not the same as the data the model was trained on. Context is dynamic, task-specific, and constructed by the engineer at runtime.

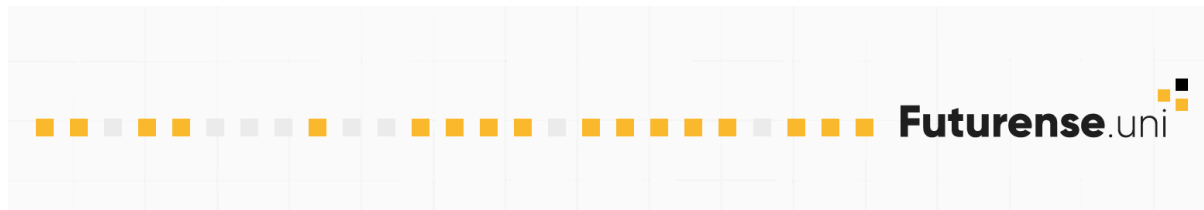
Think of a 1,000-page technical manual. Feeding all 1,000 pages to a model is wasteful and often counterproductive. The job is to retrieve exactly the 3 pages relevant to the user's current question. That selection is context engineering.

RAG (Retrieval-Augmented Generation) is one approach to context engineering, but it is a static approach. Intelligent context engineering knows which chunk, which format, and which level of detail the model needs for a given task. Wrong context produces wrong output. Precise context produces precise intelligence.

Tenet 3: Reasoning Trace

The model reasons before it responds. Modern large language models do not produce an answer instantly. They generate intermediate steps. These steps form a reasoning trace. The quality of that trace determines the quality of the output.

Reasoning traces also make failures inspectable. If a model gives a wrong answer, checking the reasoning trace shows where it went wrong. This is the foundation of debugging probabilistic systems.



Intelligence = f(Data quality, Context precision, Reasoning trace depth)

Key insight: The model is not the intelligence. The model is the engine. Data, context, and reasoning trace are the fuel, steering, and navigation. All three must be correct for the vehicle to arrive at the right destination.

3.3 Evals: Selecting and Evaluating Models

An eval is a systematic evaluation of a model's behavior against defined criteria. It is the AI equivalent of unit testing, but with statistical outcomes instead of binary pass/fail.

Many models exist at different sizes and price points. A 7-billion-parameter model costs a fraction of a 120-billion-parameter model. For simple question-and-answer tasks, the smaller model may perform equally well. For deep reasoning over 40 interdependent variables, only the larger model may produce reliable outputs.

Three categories of use cases require different model capabilities:

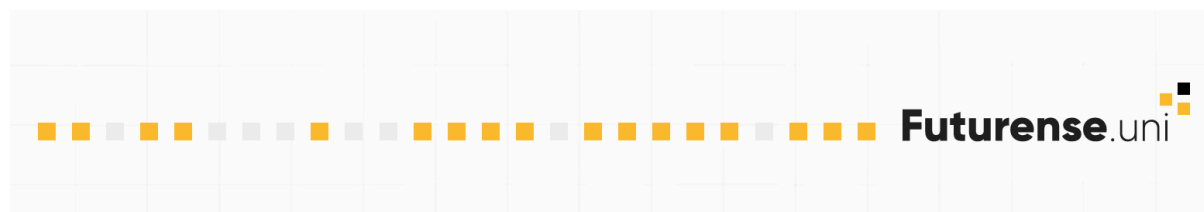
- Retrieval: Simple Q&A, keyword lookup, document summarization. Small models often sufficient.
- Routing reasoning: Finding root cause given a defined context. Medium models work well.
- Deep reasoning: Multi-variable correlation, complex inference, underwriting with 40+ variables. Large models required.

Evals measure:

- Accuracy: How often the model produces the correct answer on a defined test set
- Confidence: Whether the model's stated certainty matches its actual accuracy
- Correlation handling: Whether the model maintains correct relationships between many variables in a long context
- Drift: Whether performance degrades over time as data distributions change

Watch out: Assuming a model that works well on simple cases will work on complex ones is the most common eval mistake. Test with production-representative inputs, not toy examples.

Use Case Type	Complexity	Recommended Model Size	Key Eval Metric
FAQ / document Q&A	Low	Small (7-13B parameters)	Accuracy on fixed test set
Root cause analysis	Medium	Medium (30-70B parameters)	Reasoning trace correctness



Multi-variable inference	High	Large (70B+ parameters)	Correlation accuracy at 40+ variables
Real-time customer decisions	Very High	Large + fine-tuned	Latency + accuracy combined score

Choose model size based on task complexity, not default preference. Evaluate on representative test cases before deployment.

4. Decision Frameworks

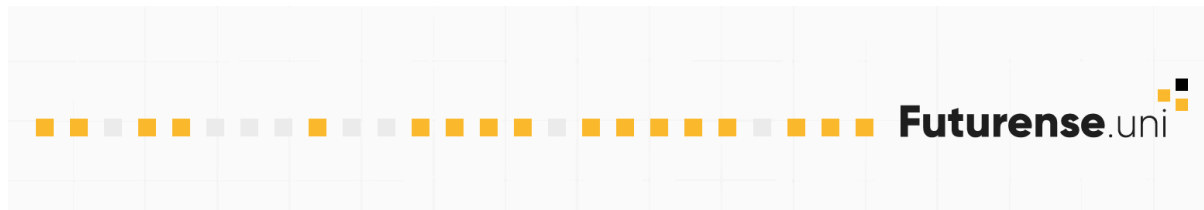
4.1 The 80/20 Rule in AI Engineering

Engineers are trained to obsess over the 20%: the model, the architecture, the algorithm choice. AI discourse amplifies this. New model announcements, benchmark comparisons, and architecture papers dominate the conversation.

The 80% is quieter but more important: data pipelines, context construction, eval design, failure mode mapping, and recovery paths. These are the things that determine whether an AI system works in production.

The 20% (model and infrastructure) is commoditized. Every engineer has access to the same models. The 80% is what differentiates a system that works from one that fails after six weeks in production.

Category	The 20% (Model Layer)	The 80% (Intelligence Layer)
What it is	Model choice, inference infrastructure, API calls	Data, context, evals, failure modes, recovery
Who controls it	Model provider (OpenAI, Anthropic, Google)	The FDE and the engineering team
Differentiation	Commoditized, available to everyone	Custom, hard to copy, proprietary value
Time investment	Low: plug-and-play via API	High: design, test, iterate continuously
Career leverage	Low: any engineer can call an API	High: few engineers build reliable intelligence



Spend 80% of engineering effort on the 80%. The model takes care of itself.

4.2 Ordinary vs. Extraordinary Problem Framing

There are two ways to receive a problem statement. The first way produces good engineers. The second produces great ones.

Example problem statement: Build an AI system that closes support tickets faster.

Ordinary framing: Build an agent that receives a ticket, queries a knowledge base, and generates a resolution in under 10 seconds. Measure success by average resolution time.

Extraordinary framing: Build a system that predicts which tickets will be raised, addresses the root cause proactively, and eliminates the ticket before the user files it. Measure success by the reduction in ticket volume.

The ordinary framing solves the stated problem. The extraordinary framing solves the upstream cause. The gap between what customers ask for and what they actually need is the space where exceptional AI engineers operate.

Key insight: Customers describe problems in terms of what exists. The FDE's job is to see what does not yet exist. That is where extraordinary value lives.

5. Code Walkthroughs

5.1 Understanding Model Output Distribution

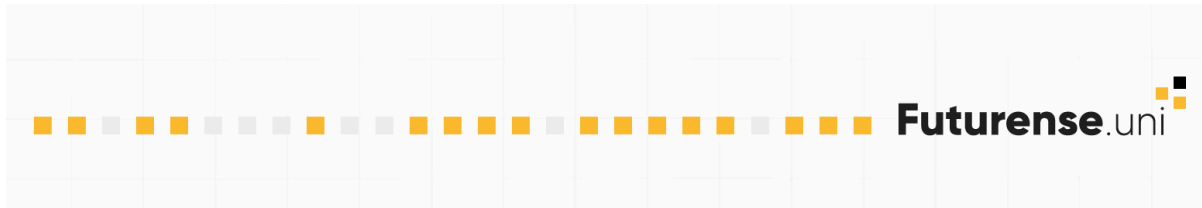
An LLM does not output a single word. It outputs a probability distribution over thousands of possible tokens, and samples from that distribution. This code demonstrates how to observe model confidence by sampling multiple times and comparing outputs.

Step 1: Query a model multiple times with the same prompt

This simulates what happens internally when temperature is above zero. Each call may return a different token sequence because sampling is probabilistic.

```
import anthropic

client = anthropic.Anthropic()
prompt = 'What is the capital of France? Answer in one word.'
```



```
results = []
for i in range(5):
    msg = client.messages.create(
        model='claude-haiku-4-5-20251001',
        max_tokens=10,
        messages=[{'role': 'user', 'content': prompt}]
    )
    results.append(msg.content[0].text.strip())

print(results)
```

Output:

```
['Paris', 'Paris', 'Paris', 'Paris', 'Paris']
```

For factual, low-ambiguity questions, the model converges. The distribution is tight. For complex or ambiguous prompts, outputs diverge. Spread in outputs signals low model confidence on that prompt.

Step 2: Measure consistency as a confidence proxy

Count how often the model returns the same answer. High agreement means high effective confidence. Low agreement means the model is uncertain, and that uncertainty should trigger a fallback.

```
from collections import Counter

counts = Counter(results)
most_common, freq = counts.most_common(1)[0]
confidence = freq / len(results)

print(f'Most common answer: {most_common}')
print(f'Consistency score: {confidence:.0%}')

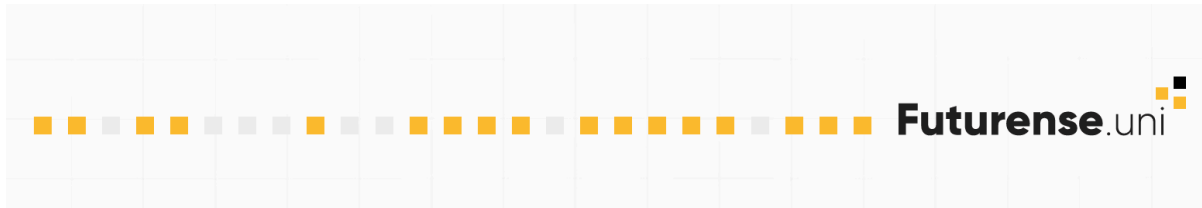
if confidence < 0.8:
    print('Low confidence: escalate or add context')
else:
    print('High confidence: answer is reliable')
```

Output:

```
Most common answer: Paris Consistency score: 100% High confidence: answer is reliable
```

This pattern is the foundation of eval design. Replace the simple prompt with a production scenario and a test set of 50 to 100 questions, and you have a working eval harness.

5.2 Context Engineering vs. Naive Retrieval



Naive retrieval passes all available documents into the model context. Intelligent context engineering selects only the relevant section. This code shows the difference in token usage and output quality.

Step 3: Calculate token cost of naive vs. targeted context

Each API call costs tokens. Fewer tokens means lower cost and better model focus. Precision in retrieval directly improves intelligence per dollar.

```
# Token estimation: 1 token ~ 4 characters

full_manual = 'A' * 400000    # 100,000 tokens (400K chars)
relevant_chunk = 'A' * 4000   # 1,000 tokens (4K chars)

naive_tokens = len(full_manual) // 4
smart_tokens = len(relevant_chunk) // 4

print(f'Naive context: {naive_tokens:,} tokens')
print(f'Smart context: {smart_tokens:,} tokens')
print(f'Reduction: {(1 - smart_tokens/naive_tokens):.0%}')
```

Output:

```
Naive context: 100,000 tokens Smart context: 1,000 tokens Reduction: 99%
```

Key insight: A 99% reduction in context size is not just a cost saving. It is a quality improvement. Models produce better outputs when context is precise rather than noisy.

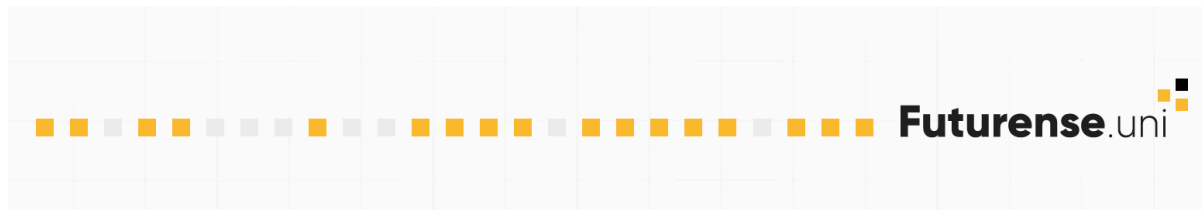
6. Case Studies

Case Study 1: Insurance Underwriting Agent

Problem: An insurance company receives applications with 40 variables each. Human underwriters take 4 hours per application. They want AI to reduce this to under 5 minutes.

Why this needs deep reasoning: Underwriting decisions depend on correlations between 40+ variables. A health condition affects risk in different ways depending on occupation, age, location, and claim history. Simple Q&A models cannot track all 40 variable interactions simultaneously.

Right approach: Use a large reasoning model. Construct context that includes the full application plus a structured risk framework. Design an eval on 500 historical applications with known outcomes. Measure correlation accuracy at 40 variables. Deploy only when accuracy exceeds 92%.



Wrong approach: Use a fast small model because it is cheaper. Test on 10 sample applications where all 10 look correct. Deploy. Discover after 3 months that the model fails when the occupation field is non-standard, causing a 35% error rate on that segment.

Result: The right approach costs 3x more per call but reduces total cost by 80% because it replaces 4 hours of human review. The wrong approach costs less per call but generates 35% rework, exceeding the original human cost.

Watch out: Testing on a small, convenient dataset is the single biggest cause of AI project failures in production. Test on representative edge cases, not just average cases.

Case Study 2: Support Ticket Elimination

Problem: A SaaS company handles 2,000 support tickets per week. They want AI to close tickets faster.

Ordinary framing result: The engineering team builds an AI agent that reads incoming tickets, queries a knowledge base, and auto-responds within 30 seconds. Resolution time drops from 4 hours to 30 seconds. Success.

Extraordinary framing result: A different engineer notices that 60% of tickets fall into 8 categories. All 8 categories map to UX friction points in the product. Instead of building a faster ticket responder, they build a system that detects when users are likely to hit those friction points and proactively shows contextual help. Ticket volume drops 55% in 90 days.

Result: The first team reduced time-per-ticket by 99%. The second team reduced ticket volume by 55%. The second outcome is more valuable, harder to copy, and the basis for a new product feature rather than a support cost reduction.

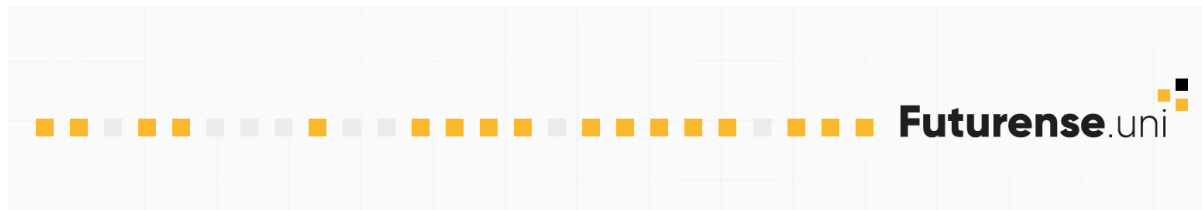
7. Common Mistakes

These are the patterns that cause AI projects to fail. Each one is avoidable once you know to watch for it.

Mistake 1: Designing for the happy path

The happy path is the flow where every input is well-formed, every API returns on time, and the model produces a sensible output. Real production systems encounter missing fields, ambiguous inputs, unusual edge cases, and data drift. A system designed only for the happy path fails in proportion to how often the path is unhappy. In production, that is often 30 to 40% of the time.

Watch out: If your system design document does not list at least 5 failure modes and their recovery paths, the design is incomplete.



Mistake 2: Treating the model as the intelligence

The model has world knowledge from pre-training. That knowledge is fixed. Your customers' data is not in the model. Your domain-specific reasoning rules are not in the model. The intelligence comes from how you construct context, how you evaluate outputs, and how you handle failures. Swapping to a newer model without improving context and evals rarely improves results.

Mistake 3: Skipping evals before deployment

Evals feel like overhead when the demo looks good. They are not overhead. They are the only way to know if the system works on real inputs. A demo uses cherry-picked inputs. Production uses everything. An eval on 100 representative cases before deployment will reveal failures that the demo concealed.

Mistake 4: Worrying about model announcements

A new model is released every few weeks. Benchmark scores change. Architecture choices evolve. Spending engineering time tracking and switching models is a low-leverage activity. The 80% (data, context, evals, failure modes) transfers across models. Build there, and model switching becomes a one-day task rather than a multi-week rewrite.

Mistake 5: Confusing hallucination with malfunction

An LLM hallucinating is not a bug in the software. It is the model behaving probabilistically in the absence of sufficient context or constraints. The engineering response is not to report it as a bug. It is to improve the context, add guardrails, and run evals to measure the rate. Humans also hallucinate. The difference is that humans have accumulated wisdom to self-correct. The engineer's job is to build that self-correction into the AI system.

8. Interview Preparation

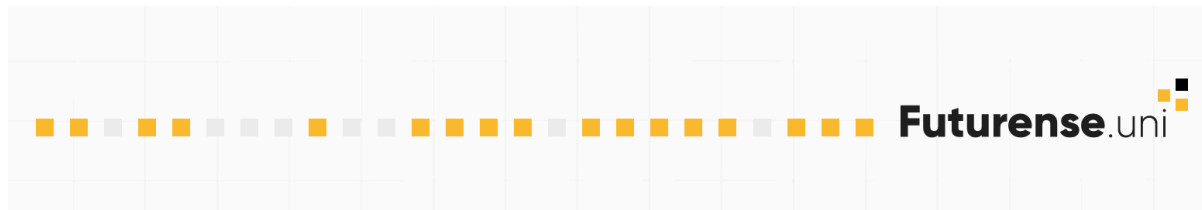
These questions appear in interviews for AI engineering, FDE, and ML engineering roles. Each answer should be precise and reference trade-offs.

Q1: What is the difference between an agent and intelligence in an AI system?

Answer: An agent is the orchestration layer. It calls tools, manages state, and coordinates steps to complete a task. Intelligence is the behavior the system produces under real conditions: accurate, consistent, robust to constraint violations, and recoverable from failure. Agents are infrastructure. Intelligence is the product. A customer pays for intelligence, not for the agent architecture that delivers it.

What they test: Whether you understand that building agents is a commodity skill, and that the differentiated work is in data, context, and evals.

Q2: How would you design an eval for an insurance underwriting AI?



Answer: Start with a labeled dataset of 500 historical applications with known correct outcomes. Define accuracy as the percentage of applications where the AI's decision matches the ground truth. Segment by variable count: test separately on applications with 15, 30, and 40+ variables, since complexity degrades performance differently. Measure confidence calibration: does the model's stated confidence match its actual accuracy? Set deployment thresholds, for example 92% accuracy on 40-variable cases. Monitor for drift every 30 days after deployment.

What they test: Whether you can design a structured eval rather than just run informal demos.

Q3: How do you handle hallucination in a production AI system?

Answer: Hallucination is a signal that the model lacks sufficient context or constraints for the task. The fix is not to swap models. It is to improve context precision, add structured output formats to reduce free-form generation, implement confidence thresholds that trigger human review on low-confidence outputs, and run evals to measure hallucination rate per task category. The goal is to reduce rate to an acceptable level and build recovery paths for the cases that remain.

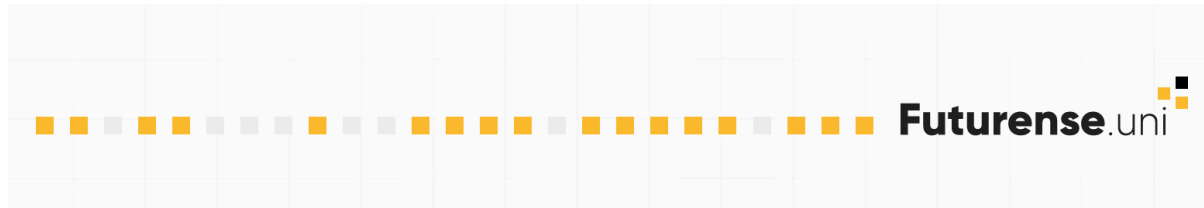
What they test: Maturity in thinking about AI limitations as engineering constraints rather than product failures.

Q4 (Code challenge): Write a function that measures consistency of a model's output as a proxy for confidence.

```
from collections import Counter
import anthropic

def eval_confidence(prompt, n_samples=10, threshold=0.8):
    client = anthropic.Anthropic()
    outputs = []
    for _ in range(n_samples):
        r = client.messages.create(
            model='claude-haiku-4-5-20251001',
            max_tokens=50,
            messages=[{'role': 'user', 'content': prompt}]
        )
        outputs.append(r.content[0].text.strip())
    top, freq = Counter(outputs).most_common(1)[0]
    confidence = freq / n_samples
    return {'answer': top, 'confidence': confidence,
            'reliable': confidence >= threshold}

result = eval_confidence('What is 2 + 2?')
print(result)
```



Output:

```
{'answer': '4', 'confidence': 1.0, 'reliable': True}
```

This pattern scales directly to production evals. Replace the prompt with a test case from your labeled dataset and run across hundreds of examples.

9. Practice Problems

9.1 MCQs

Q1. An engineer is asked to build an AI system that auto-responds to support tickets. What is the most valuable problem framing?

- A) Build an agent that retrieves relevant answers and responds in under 10 seconds.
- B) Build an agent that triages tickets by severity and assigns them to the right team.
- C) Build a system that detects patterns in incoming tickets and eliminates root causes proactively.
- D) Build a dashboard that shows ticket volume trends for the support manager.

Answer: C. Option C reframes the problem from ticket resolution to ticket elimination. This is the extraordinary framing. Options A and B solve the stated problem (faster response). Option D is analytical, not AI engineering.

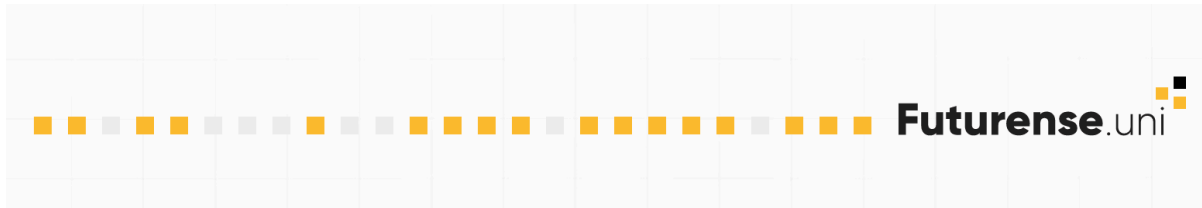
Q2. Which of the following best describes the difference between speed and velocity in AI engineering?

- A) Speed is how fast a model generates tokens. Velocity is the number of API calls per second.
- B) Speed is how quickly code is shipped. Velocity is speed plus strategic direction of impact.
- C) Speed is a property of large models. Velocity is a property of small, fast models.
- D) Speed and velocity are synonymous in software engineering.

Answer: B. Option B is correct. Speed measures output rate. Velocity adds direction: are you building the right thing fast? Options A and C confuse the terms with model or system performance metrics.

Q3. A model is producing inconsistent answers on a 40-variable underwriting use case. What is the most likely cause?

- A) The model has insufficient pre-training data.
- B) The context window is not large enough to hold all 40 variables.
- C) The model size is too small to track correlations across 40 variables.
- D) The API rate limit is causing incomplete responses.



Answer: C. Option C is correct. Small models struggle to maintain coherent correlation tracking over many variables. This is a model capability limit, not a context window or API issue. Option B is possible but secondary: the primary issue is reasoning depth.

Q4. What is the primary purpose of an eval in AI engineering?

- A) To demonstrate to stakeholders that the AI system is working.
- B) To measure model behavior against defined criteria on representative test cases.
- C) To compare the cost of different LLM API providers.
- D) To generate synthetic training data for fine-tuning.

Answer: B. Option B is correct. An eval is a systematic measurement of model behavior under defined conditions. Option A describes a demo, not an eval. Options C and D are separate activities.

Q5. An AI support ticket agent performs well in testing on 10 sample tickets. After deployment, it fails on 35% of tickets. What most likely caused this?

- A) The model was updated by the provider between testing and deployment.
- B) The test set was too small and did not represent the full distribution of production inputs.
- C) The agent's tool-calling logic has a race condition under high load.
- D) The API context window was exceeded in production.

Answer: B. Option B is correct. Testing on 10 samples is insufficient. Production inputs cover a far wider distribution including edge cases not present in the small test set. This is the most common cause of post-deployment failures.

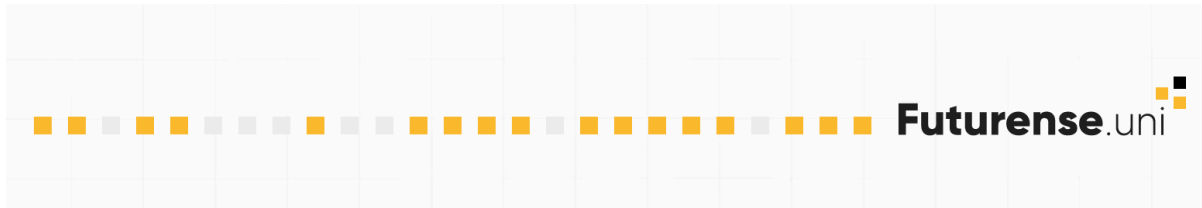
Q6. Which of the following is inside the 80% that an FDE controls?

- A) The model's transformer architecture.
- B) The number of parameters in the base model.
- C) The context constructed and passed to the model at inference time.
- D) The hardware infrastructure used for model training.

Answer: C. Option C is correct. Context construction is fully controlled by the engineer. Options A, B, and D belong to the model provider and are outside engineer control.

Q7. A model produces different answers to the same factual question on repeated calls. What does this indicate?

- A) A bug in the inference server.
- B) The model is running at temperature > 0, generating probabilistic outputs.
- C) The context window is being reset between calls.
- D) The API key is being shared across multiple users.



Answer: B. Option B is correct. Temperature above zero means the model samples from a probability distribution rather than returning the single highest-probability token. This is expected behavior, not a bug.

Q8. What is the role of a Forward Deployed Engineer?

- A) To manage model training pipelines in data centers.
- B) To write Python wrappers around LLM APIs for internal use.
- C) To deliver AI intelligence in the field by solving real customer problems with engineering judgment and domain fluency.
- D) To do sales engineering for AI products and prepare demo scripts.

Answer: C. Option C is correct. An FDE is a field engineer who solves extraordinary problems for customers using AI. They combine technical depth, communication skill, and system design judgment. They are not a sales role (D) or a pure infrastructure role (A, B).

Q9. Read this code and identify what it measures: `counts = Counter(results) top, freq = counts.most_common(1)[0] confidence = freq / len(results)`

- A) The semantic similarity between different model outputs.
- B) The proportion of outputs that match the most common response, used as a confidence proxy.
- C) The token count of the most frequent response.
- D) The latency distribution across multiple API calls.

Answer: B. Option B is correct. The code counts how often the same answer appears across multiple runs. High frequency means high consistency, which is used as a proxy for model confidence on that prompt.

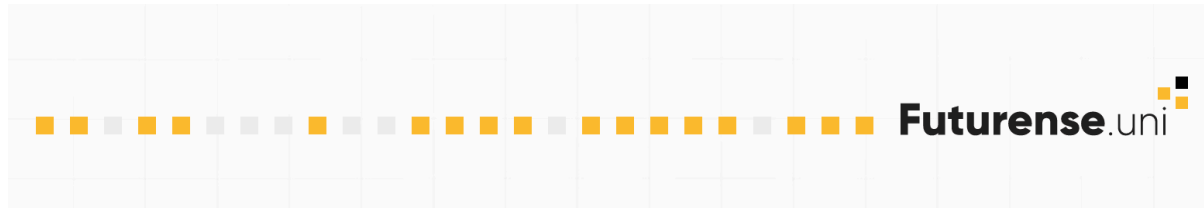
Q10. An LLM produces a wrong answer on a complex multi-variable query. A colleague says it is 'just hallucinating' and recommends switching models. What is the most appropriate response?

- A) Agree and switch to the latest model immediately.
- B) Run the same query 10 times and report the average quality.
- C) Investigate whether the context was precise enough, run an eval on representative cases, and check if a larger model with better reasoning is needed before recommending a fix.
- D) Add a retry loop to regenerate the answer until it looks correct.

Answer: C. Option C is correct. The right response is systematic investigation: check context quality, run evals, and assess whether model size is the bottleneck. Option A is a reflex, not an engineering decision. Option D treats hallucination as a bug rather than a probabilistic behavior.

9.2 Assignments

Assignment 1: Design an Eval for a Banking AI Use Case



Goal: Apply eval design skills to a real domain.

Scenario: A bank wants to use AI to assess loan applications. Each application has 25 variables: credit score, income, employment history, debt-to-income ratio, asset value, and 20 others. The AI must classify each application as Approved, Conditional, or Rejected with a reason.

Requirements:

- Define the test dataset: how many samples, what distribution of outcomes, and how to source labeled ground truth.
- Choose 3 metrics to measure and explain why each matters for this use case.
- Define deployment thresholds: at what accuracy level would you recommend going live?
- Describe one failure mode and its recovery path.

Submit: A 1-page structured document with your eval design. Distinguish between what you would measure, how you would measure it, and what you would do if the threshold is not met.

Strong answer: Addresses edge cases (applicants with no credit history), includes confidence calibration, and specifies the recovery path (escalate to human underwriter) with a triggering condition. Weak answer: Lists accuracy as the only metric and sets a threshold without justifying it.

Assignment 2: Reframe a Ticket Problem as an Extraordinary Problem

Goal: Practice the ordinary-to-extraordinary problem framing shift.

Scenario: An e-commerce company handles 5,000 returns per month. 70% of returns are logged as 'item did not match description.' Leadership wants an AI to process return requests faster.

Requirements:

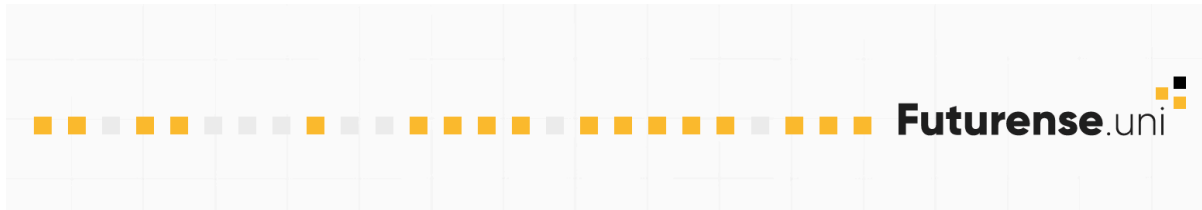
- Write the ordinary framing: what the problem looks like if you take the request at face value.
- Write the extraordinary framing: what upstream cause, if addressed, would reduce the problem rather than just speed it up.
- For the extraordinary framing, describe what data you would need and what AI capability would solve it.

Submit: Two parallel framings of no more than 200 words each. Then 150 words on the data and AI capability required for the extraordinary solution.

Strong answer: Identifies that product descriptions are the upstream cause, proposes an AI system that flags mismatches between product images, descriptions, and historical return reasons before listing, and specifies the data needed (image-description pairs, historical return data). Weak answer: Proposes an AI that only speeds up return form processing.

Assignment 3: Build a Simple Eval Harness

Goal: Write working code that measures model output consistency.



Scenario: Build a function that takes a list of question-answer pairs and returns an accuracy report. Use any LLM API.

Requirements:

- Function takes: a list of dicts with 'question' and 'expected_answer' keys.
- For each question, call the model and compare the response to the expected answer.
- Return: total accuracy, list of failed cases, and average consistency score across 3 runs per question.

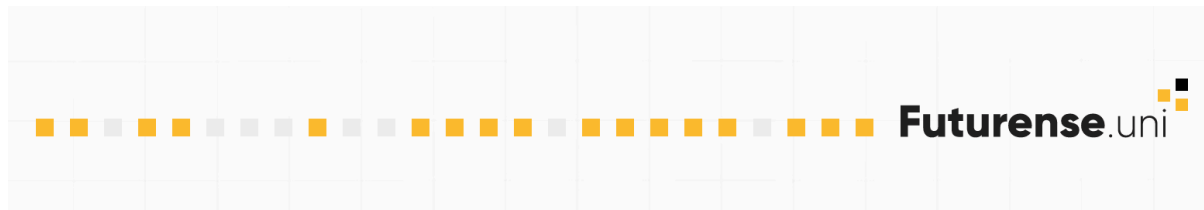
Submit: Working Python code with output from running it on at least 5 test cases.

Grading: Strong submission includes the consistency score (3-run agreement) not just binary correct/wrong, handles API errors gracefully, and uses clear variable names.

10. Further Learning

These topics build directly on this guide. Approach them in order.

- System Design for AI: How to structure an AI system that handles failure, scales context, and maintains intelligence over time. Focus on constraint-first design.
- Context Engineering: Advanced retrieval beyond RAG. Semantic chunking, re-ranking, and dynamic context construction based on query type.
- Eval Frameworks: DeepEval, RAGAS, and custom eval harnesses. How to automate eval runs and track performance over model and data versions.
- Fine-tuning: When to fine-tune vs. prompt-engineer. LoRA and PEFT for domain adaptation without full retraining.
- Knowledge Graphs: How structured knowledge enhances context quality. Useful for domains where entity relationships matter more than raw text retrieval.
- AI Failure Mode Taxonomy: Hallucination, calibration failure, context overflow, reasoning shortcut, and data drift. How to detect and recover from each.
- Patents as Engineering Reference: Reading AI patents reveals the design thinking behind systems. Search for patents in knowledge graph-driven experience design as a starting point.



11. Cheatsheet

Keep this section for quick reference during design reviews and interview prep.

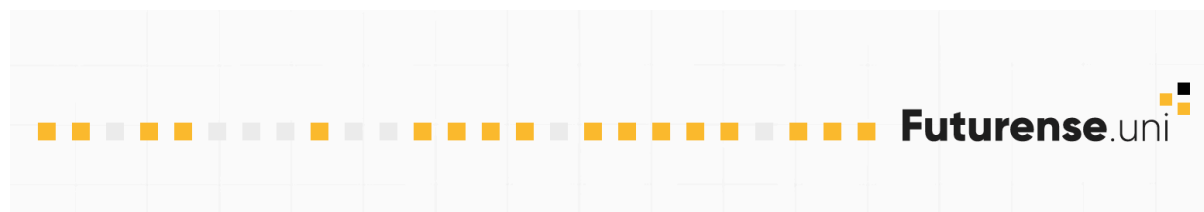
Key Definitions

Term	Definition
Speed	Rate of output, no direction
Velocity	Rate of output with strategic direction
Deterministic system	Same input always produces same output
Probabilistic system	Same input produces distribution of outputs
Agent	Orchestration layer that calls tools and takes actions
Intelligence	Reliable, accurate, failure-aware behavior that customers pay for
Context	Task-specific data provided to the model at inference time
Eval	Systematic measurement of model behavior against defined criteria
Unit of Intelligence	Behavior, under specified conditions, with failure modes and recovery paths defined
Data drift	Change in input distribution over time that degrades model performance
Hallucination	Model generates plausible but incorrect output due to insufficient context or constraints

Reference these definitions in design discussions and written evaluations.

When to Use What

Situation	Right Approach
Simple FAQ use case	Small model, precise context, high test coverage



Complex multi-variable reasoning	Large reasoning model, structured context, deep eval
Model producing inconsistent outputs	Improve context precision, add constraints, run consistency eval
Hallucination in production	Tighten context, add guardrails, set confidence threshold for human escalation
New model released	Run existing evals on new model before switching; check 80% transfer
Stakeholder asks for agent	Build agent and intelligence layer; demo intelligence, not just orchestration

Use situation-response mapping to avoid common wrong turns.

Formulas

Velocity = Speed + Direction of Impact

Intelligence = f(Data quality, Context precision, Reasoning trace depth)

Consistency confidence = frequency(top_answer) / total_samples

Token reduction = 1 - (smart_tokens / naive_tokens)

Unit of Intelligence = Behavior + Conditions + Eval + Failure modes + Recovery paths

Golden Rules

- The model is not the intelligence. The intelligence is what you build around it.
- Design for failure first. The happy path takes care of itself.
- Context precision beats context volume. Send what the model needs, nothing more.
- Eval before deploying. Always. Ten test cases is not an eval.
- The 80% (data, context, evals, failure modes) transfers across models. Build there.
- Extraordinary problems require seeing what does not yet exist. Customers only see what already does.
- Agents are orchestration. Intelligence is the product.
- Velocity includes direction. Building the wrong thing faster is not progress.

Common Commands

```
# Install Anthropic SDK
```

```
pip install anthropic
```

```
# Basic API call
```

```
import anthropic
```

```
client = anthropic.Anthropic()
```

```
msg = client.messages.create(
```

```
    model='claude-haiku-4-5-20251001',
```

```
    max_tokens=100,
```

```
    messages=[{'role': 'user', 'content': 'Your prompt here'}]
```

```
)
```

```
print(msg.content[0].text)
```